

SAiDL Summer Assignment 2026

Sparsity & Optimization

By Aayush Kushwaha

Abstract

We study parameter-efficient fine-tuning (PEFT) methods for adapting pretrained language models. We compare LoRA, AdaLoRA, and Sparse Low-Rank Adaptation (SoRA) on the CoLA task from the GLUE benchmark using DeBERTaV3-base as the backbone. We analyze performance, trainable parameter count, effective rank, and training efficiency. We provide a mathematical comparison between proximal gradient descent and SGD with ℓ_1 subgradients for the SoRA gating optimization, and extend adaptive low-rank adaptation to recurrent architectures (xLSTM, Mamba).

1 Part 1: Comparing LoRA, AdaLoRA, and SoRA

1.1 Background

1.1.1 LoRA: Low-Rank Adaptation

LoRA freezes the pretrained weight matrix $W_0 \in \mathbb{R}^{k \times d}$ and learns a low-rank decomposition of the update:

$$W = W_0 + BA, \quad A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(k, d) \quad (1)$$

The rank r is fixed as a hyperparameter. Only A and B are trained, reducing trainable parameters from $k \times d$ to $r(k + d)$.

1.1.2 AdaLoRA: Adaptive Budget Allocation

AdaLoRA parameterizes the weight update using a singular value decomposition:

$$\Delta W = P\Lambda Q, \quad P \in \mathbb{R}^{k \times r}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_r), \quad Q \in \mathbb{R}^{r \times d} \quad (2)$$

It dynamically prunes singular values based on an importance score, reallocating rank budget across layers during training.

1.1.3 SoRA: Sparse Low-Rank Adaptation

SoRA introduces a learnable gating vector $\mathbf{g} \in \mathbb{R}^r$ with ℓ_1 regularization:

$$\Delta W = B \cdot \text{diag}(\mathbf{g}) \cdot A \quad (3)$$

The ℓ_1 penalty on $\mathbf{g}$ drives gate entries to zero via proximal gradient descent, automatically determining the effective rank without manual selection.

1.2 Experimental Setup

- **Backbone:** DeBERTaV3-base (86M parameters)
- **Task:** CoLA (Corpus of Linguistic Acceptability) from GLUE benchmark
- **Metric:** Matthews Correlation Coefficient (MCC)
- **LoRA rank:** $r = 8$ (fixed)
- **AdaLoRA initial rank:** $r_{\text{init}} = 12$, target rank budget varies
- **SoRA initial rank:** $r = 8$, λ controls sparsity
- **Epochs:** 20

1.3 Results

1.3.1 (a) Performance on CoLA

Method	MCC	Accuracy
LoRA ($r = 8$)	0.3574	74.98%
AdaLoRA	0.3919	76.22%
SoRA	0.3711	72.18%

1.3.2 (b) Number of Trainable Parameters

Method	Total Params	Trainable Params	% Trainable
LoRA ($r = 8$)	86M	1.03M	0.55%
AdaLoRA	86M	1.25M	0.67%
SoRA	86M	0.44M	0.24%

1.3.3 (c) Effective Rank After Training

The effective rank measures how many dimensions of the low-rank update are actually utilized after training. For LoRA, the effective rank equals the fixed rank r . For AdaLoRA, it corresponds to the number of retained (non-pruned) singular values. For SoRA, the effective rank is given by the number of active gates,

$$r_{\text{eff}} = \|\mathbf{g}\|_0.$$

Method	Initial Rank	Effective Rank	Rank Reduction
LoRA ($r = 8$)	8	8 (fixed)	0%
AdaLoRA	12	6.8	43.3%
SoRA	8	1	87.5%

1.3.4 (d) Training Efficiency

Method	Time/Epoch (s)	Total Time (s)
LoRA ($r = 8$)	185.3s	3706.9s
AdaLoRA	186.2s	3724.5s
SoRA	186.2s	3725.9s

Therefore, time taken is approximately same for all the methods.

1.4 Analysis: Fixed vs. Adaptive Rank

2 Part 2: Proximal Gradient vs. SGD with ℓ_1 Subgradients

2.1 Problem Setup

In SoRA, the gating vectors $\mathbf{g}^{(k)}$ are optimized via the ℓ_1 -regularized objective:

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \sum_{k=1}^K \|\mathbf{g}^{(k)}\|_1 \quad (4)$$

where $\mathcal{L}_0$ is the task loss and $\lambda > 0$ controls sparsity.

2.2 (a) SGD Update Rule with ℓ_1 Subgradients

2.2.1 Derivation

The ℓ_1 norm is non-differentiable at zero. We use the subdifferential:

$$\partial\|g_i\|_1 = \partial|g_i| = \begin{cases} \{+1\} & \text{if } g_i > 0 \\ \{-1\} & \text{if } g_i < 0 \\ [-1, +1] & \text{if } g_i = 0 \end{cases} \quad (5)$$

Choosing the subgradient $s_i \in \partial|g_i|$ (commonly $s_i = \text{sign}(g_i)$ with $\text{sign}(0) = 0$), the SGD update for gate g_i at step t is:

$$g_i^{(t+1)} = g_i^{(t)} - \eta \left(\left. \frac{\partial \mathcal{L}_0}{\partial g_i} \right|_{g_i^{(t)}} + \lambda \cdot \text{sign}(g_i^{(t)}) \right) \quad (6)$$

where η is the learning rate.

2.3 (b) Mathematical Comparison: SGD vs. Proximal Update

2.3.1 Proximal Gradient (Soft-Thresholding) Update

The proximal operator for $\lambda \|\cdot\|_1$ is the soft-thresholding operator:

$$\text{prox}_{\eta\lambda\|\cdot\|_1}(u) = \text{sign}(u) \cdot \max(|u| - \eta\lambda, 0) \quad (7)$$

Applied after a gradient step on the smooth part $\mathcal{L}_0$:

$$g_i^{(t+1)} = \text{sign}(\tilde{g}_i) \cdot \max(|\tilde{g}_i| - \eta\lambda, 0), \quad \tilde{g}_i = g_i^{(t)} - \eta \frac{\partial \mathcal{L}_0}{\partial g_i} \quad (8)$$

2.3.2 Side-by-Side Comparison

Table 1: Comparison of update rules for ℓ_1 -regularized gating optimization.

Property	SGD + Subgradient	Proximal (Soft-Thresholding)
Update rule	$g_i \leftarrow g_i - \eta(\nabla_{g_i} \mathcal{L}_0 + \lambda \text{sign}(g_i))$	$g_i \leftarrow \text{sign}(\tilde{g}_i) \max(\tilde{g}_i - \eta\lambda, 0)$
Exact sparsity	No — values oscillate around zero but rarely hit exactly zero	Yes — values are explicitly set to zero when $ \tilde{g}_i < \eta\lambda$
Behavior at $g_i = 0$	Subgradient $\text{sign}(0) = 0$, so g_i stays at zero only if $\nabla \mathcal{L}_0 = 0$	g_i remains zero whenever $ \nabla \mathcal{L}_0 < \lambda$
Convergence	$O(1/\sqrt{t})$ — slower, non-smooth	$O(1/t)$ — faster, leverages problem structure
Stability	Can oscillate near zero	Stable, monotonically decreasing objective

2.3.3 Key Mathematical Difference

Consider g_i close to zero after the gradient step, i.e., $|\tilde{g}_i|$ is small:

- **Proximal:** If $|\tilde{g}_i| < \eta\lambda$, the output is *exactly* zero. This creates a “dead zone” that induces true sparsity.
- **SGD:** The update $g_i - \eta\lambda \text{sign}(g_i)$ can *overshoot* zero.

Figure 1: Trajectory of a single gate value g_i under SGD (oscillates around zero) vs. proximal gradient (converges to exact zero).

2.4 (c) Effect of Subgradient Choice

At $g_i = 0$, the subdifferential is $\partial|g_i| = [-1, +1]$. Any $s_i \in [-1, 1]$ is a valid subgradient. We analyze whether this choice affects the comparison.

2.4.1 Common Choices

2.4.2 (c) Effect of Subgradient Choice

At $g_i = 0$, the subdifferential of the ℓ_1 regularizer is

$$\partial|g_i| = [-1, +1].$$

Three common choices are considered:

Choice 1: $s_i = 0$ **when** $g_i = 0$ The SGD update becomes

$$g_i^{(t+1)} = 0 - \eta \nabla_{g_i} L_0.$$

The ℓ_1 regularizer is ignored at zero, and the gate is updated solely by the task-loss gradient. Once a gate reaches zero, only the task gradient can move it away. Consequently, gates can escape zero easily and sparsity is not maintained.

Choice 2: $s_i = +1$ **when** $g_i = 0$ The SGD update becomes

$$g_i^{(t+1)} = 0 - \eta (\nabla_{g_i} L_0 + \lambda).$$

The gate is pushed to $-\eta\lambda$. On the next iteration, $\text{sign}(g_i) = -1$, causing the regularization term to push the gate back toward positive values. This leads to sign oscillation around zero,

$$\dots, +\varepsilon, -\varepsilon, +\varepsilon, -\varepsilon, \dots$$

rather than stable sparsity.

Choice 3: Random $s_i \in [-1, +1]$ A random subgradient is selected whenever $g_i = 0$:

$$s_i \sim \mathcal{U}(-1, +1).$$

This produces stochastic oscillation near zero, exhibiting behavior similar to Choice 2 but with a random direction at each update.

Key Insight None of these subgradient choices produce exact sparsity. The probability that a gate remains exactly at zero after an SGD update is

$$P(g_i^{(t)} = 0 \mid g_i^{(t-1)} \neq 0) = P(\nabla_{g_i} L_0 = -\lambda s_i) = 0.$$

In continuous optimization, the probability that the task gradient exactly cancels the ℓ_1 subgradient is zero. Therefore, regardless of the subgradient choice, SGD almost never produces exact sparsity.

The proximal method avoids this issue through the soft-thresholding operator

$$g_i = \text{sign}(\tilde{g}_i) \max(|\tilde{g}_i| - \eta\lambda, 0),$$

which explicitly projects onto a sparse solution. Consequently, exact zeros are guaranteed whenever

$$|\tilde{g}_i| < \eta\lambda.$$

2.4.3 Analysis

Proposition 1. *The choice of subgradient at $g_i = 0$ does not affect the **proximal** update, which is uniquely defined by the proximal operator. However, it does affect the SGD update.*

Proof. □

Remark 1. *With $s_i = 0$ at $g_i = 0$, the SGD update ignores the regularizer entirely when the gate is at zero, meaning once a gate reaches zero, it can only be moved away by the task loss gradient. With $s_i \neq 0$, the gate may be pushed through zero, leading to sign oscillation. Neither choice produces exact sparsity reliably, unlike the proximal method.*

Table 2: Effect of subgradient choice on SoRA training (CoLA, DeBERTaV3-base).

Optimization Method	MCC	Effective Rank	Exact Zeros in g
Proximal (soft-thresholding)	0.3711	1.0	87.50%
SGD, $\text{sign}(0) = 0$	0.3384	8.0	0.00%
SGD, $\text{sign}(0) = +1$	0.3319	8.0	0.00%
SGD, random subgradient	0.3347	8.0	0.00%

3 Part 3: Extending Adaptive Low-Rank Adaptation to Recurrent Architectures

3.1 Proposed Approach for xLSTM

The extended Long Short-Term Memory (xLSTM) architecture introduces exponential gating in the sLSTM variant, replacing the traditional sigmoid input gate with an exponential activation:

$$i_t = \exp(W_i[x_t, h_{t-1}] + b_i)$$

This produces unbounded gate activations that enable finer-grained memory control. Each sLSTM cell contains four projection matrices — input gate (W_i), forget gate (W_f), output gate (W_o), and cell candidate (W_z) — each mapping from $\mathbb{R}^{2d} \rightarrow \mathbb{R}^d$ (concatenation of input and hidden state). Additionally, each layer wraps the cell with input/output projections ($W_{in}, W_{out} \in \mathbb{R}^{d \times d}$).

SoRA injection strategy. We apply SoRA adapters to all six linear projections per layer:

$$\Delta W_k = B_k \cdot \text{diag}(g_k) \cdot A_k, \quad k \in \{i, f, o, z, in, out\}$$

A key difference from the Transformer setting is that these projections are applied at every timestep within the recurrent scan. This means the SoRA adapters are shared across timesteps (parameter sharing), making the effective number of adapted forward passes $L \times K$ per layer (where L is sequence length and $K = 6$ is the number of target modules), compared to just K in a Transformer layer. This temporal reuse amplifies the impact of each adapter, motivating lower initial ranks and stronger sparsity pressure than in the Transformer case.

The gating vectors g_k are optimized via proximal gradient descent with ℓ_1 regularization, identical to the Transformer SoRA formulation. The proximal operator is applied per-batch after the AdamW update on A_k, B_k :

$$g_k \leftarrow S_{\eta\lambda}(g_k - \eta \nabla_{g_k} L_0)$$

where

$$S_\tau(x) = \text{sign}(x) \cdot \max(|x| - \tau, 0)$$

is the soft-thresholding operator.

Architectural consideration. The gate projections (W_i, W_f, W_o, W_z) have input dimension $2d$ (concatenated $[x_t, h_{t-1}]$), while the layer projections (W_{in}, W_{out}) have input dimension d . SoRA automatically handles this heterogeneity since each adapter’s A_k matrix matches the input dimension of its target module.

We adopt a two-phase training strategy: (1) a brief pretraining phase (3 epochs) of the full base model to establish meaningful hidden representations, followed by (2) freezing the base

model and training only the SoRA adapters with proximal sparsity pressure. This is necessary because, unlike DeBERTa, the xLSTM model is initialized from scratch and requires a warm-start for the adapters to have meaningful gradients.

3.2 Proposed Approach for Mamba

The Mamba architecture is built on the Structured State Space Model (S4) framework, where each layer implements a discretized linear dynamical system:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t$$

where

$$\bar{A} = \exp(\Delta A)$$

and

$$\bar{B} = (\Delta A)^{-1} (\exp(\Delta A) - I) \cdot \Delta B$$

are the discretized state transition and input matrices. Mamba’s key innovation is making B , C , and the discretization step Δ input-dependent (selective scan), enabling content-based reasoning.

Each Mamba layer contains four key projections:

- $W_{in} \in \mathbb{R}^{d \times ed}$: Input expansion (expand factor $e = 2$)
- $W_{out} \in \mathbb{R}^{ed \times d}$: Output contraction
- $W_B, W_C \in \mathbb{R}^{d \times n}$: State-space input/output projections (n = state dimension)

SoRA injection strategy. We target the input/output projections and the SSM parameter projections:

$$\Delta W_k = B_k \cdot \text{diag}(g_k) \cdot A_k, \quad k \in \{in, out, B, C\}$$

The SSM projections (W_B, W_C) are particularly interesting for SoRA because they directly control the information flow into and out of the state space. Pruning gate dimensions in these projections effectively reduces the rank of the state-space interaction, providing a principled way to discover the minimal state-space dimensionality needed for the task.

Key differences from Transformer SoRA. Sequential state dependency. In SSMs, each timestep’s output depends on the entire history via the hidden state h_t . Adapter perturbations at early timesteps propagate through the entire sequence, making the optimization landscape more sensitive than in Transformers where attention allows direct long-range connections. We address this by using a lower learning rate (2×10^{-4}) and aggressive gradient clipping (max norm 1.0).

Stability constraints. The state transition matrix $\bar{A}$ must have eigenvalues with magnitude ≤ 1 for stability. While SoRA does not directly modify $\bar{A}$, perturbations to the input projection W_{in} can indirectly affect the effective dynamics. We initialize A_{log} with $\log(\text{Uniform}(0.1, 1.0))$ and clamp to $[-\infty, 3.0]$ to ensure stable decay rates.

Fewer target modules. Each SSM layer has only 4 target projections compared to 6 in xLSTM, resulting in fewer total SoRA adapters and a more compact adaptation footprint.

The same two-phase training strategy (pretrain base $\rightarrow$ freeze and train SoRA) and proximal optimization are applied, with the sparsity pressure ramped over the second half of training to allow the adapters to first learn useful directions before pruning begins.

Model	PEFT Method	MCC	Trainable %	Eff. Rank
xLSTM	LoRA ($r = 8$)	0.0858	0.39%	8
xLSTM	SoRA	0.1198	0.40%	4.8
Mamba	LoRA ($r = 8$)	0.0612	0.32%	8
Mamba	SoRA	0.0518	0.33%	5.1

References

- [1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [2] Q. Zhang et al., “AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning,” *ICML*, 2023.
- [3] N. Ding et al., “Sparse Low-Rank Adaptation of Pre-trained Language Models,” *EMNLP*, 2023.
- [4] M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” *arXiv:2405.04517*, 2024.
- [5] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [6] N. Parikh and S. Boyd, “Proximal Algorithms,” *Foundations and Trends in Optimization*, 1(3):127–239, 2014.
- [7] P. He et al., “DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing,” *ICLR*, 2023.
- [8] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 9(8):1735–1780, 1997.
- [9] A. Gu et al., “Efficiently Modeling Long Sequences with Structured State Spaces,” *ICLR*, 2022.